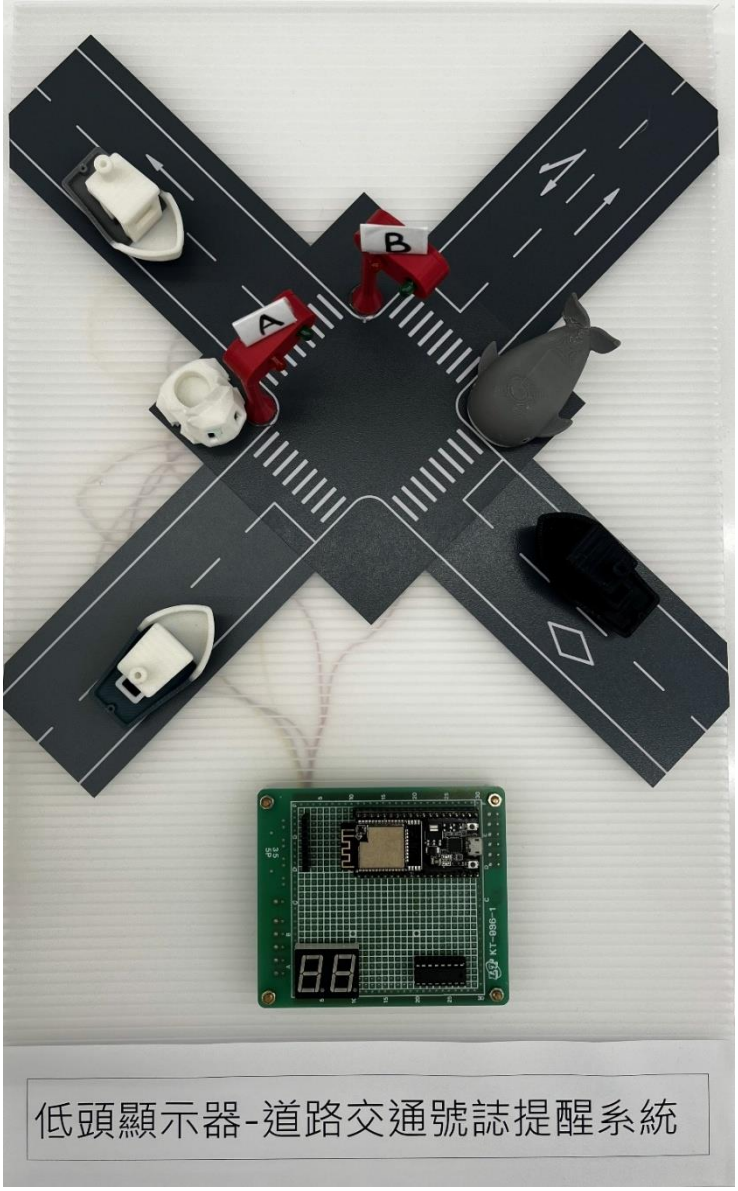
全國高級中等學校專業群科 115 年專題實作及創意競賽

「創意組」作品說明書



群　　別：電機與電子群

作品名稱：低頭顯示器-道路交通號誌提醒系統

關 鍵 詞：紅綠燈提醒、交通安全、IoT智慧交通

目錄

壹、創意動機及目的 1

一、創意動機.....1

二、創意目的.....1

貳、作品特色與創意特質 2

一、從理論到硬體 2

二、從邏輯到軟體 2

三、從技術到智慧城市 2

四、從實作到社會關懷 2

參、創意發想與設計過程 2

一、創意發想.....2

二、設計過程.....3

(一)手機端藍芽接收不需配對 3

(二)控制成本達成系統需求.....3

(三)手機 App 程式設計.....3

肆、設計相關原理 3

一、元件清單.....4

(一)微控制器 4

(二)電源.....4

(三)輸出元件.....4

二、電子電路設計（硬體） 5

三、系統程式設計（通訊與控制） 6

四、BLE 通訊協定與封包設計 6

五、使用者操作與應用流程 8

伍、作品功用與操作方式 9

一、硬體模擬架構 9

二、手機 APP 功能 9

陸、製作歷程說明 10

一、 硬體功能測試.....10

(一) 紅綠燈模擬與同步測試.....10

(二) 顯示驅動測試.....10

(三) 組裝階段驗證.....10

二、 電路設計及焊接.....11

三、 手機 APP 程式設計與軟體測試.....11

四、 作品平台整合.....12

柒、附錄.....14

一、作品分工表 14

二、競賽日誌.....15

圖目錄

圖 1 取材自民視新聞網 youtube 頻道 1
圖 2 系統架構圖 4
圖 3 Android 接收程式解析架構.....8
圖 4 作品硬體及軟體展示 9
圖 5 電路接線測試 11
圖 6 完成電路設計及焊接 11
圖 7 手機 App 程式開發.....12
圖 8 Android 程式開發畫面.....12
圖 9 號誌燈桿 3D 設計.....13
圖 10 號誌燈桿 3D 列印.....13
圖 11 硬體架構平台組裝完成 14

低頭顯示器-道路交通號誌提醒系統

壹、創意動機及目的

一、創意動機

我們在路上經常看到汽機車停等紅燈時，有些機車騎士或開車者專注滑手機而忘記看紅燈已轉換成綠燈，因此造成交通阻塞或意外；也有後車因前車未開，不耐煩的狂按喇叭，甚至因此而使得有「路怒症」患者與駕駛雙方發生衝突，新聞屢見不鮮(圖 1)。

圖 1 取材自民視新聞網 youtube 頻道

在現代交通環境中，用路人習慣仰賴視覺判讀燈號，然而，隨著「低頭族」增加或因注意力分散，駕駛人常忽略燈號切換的瞬間，潛藏安全危機。若能結合普及的無線通訊技術，主動提供燈號資訊，將能有效預防交通事故。

二、創意目的

我們設計了一套基於 BLE 藍芽技術的紅綠燈提醒系統，目的是讓交通號誌能「主動說話」。系統透過 ESP32 模擬交通燈號，將剩餘秒數以廣播方式發送，使用者只需透過手機即可接收資訊，完全不需要繁瑣的藍芽配對。這種設計具有省電與反應快速的優勢，能有效協助駕駛人掌握燈號切換時機，進而解決「低頭族」因滑手機可能造成的交通危害。

貳、作品特色與創意特質

我們都知道車子有所謂的「抬頭顯示器」，就是將行車速度的相關資訊顯示於汽車擋風玻璃上，我們想到本專題是為了解決「低頭族」開車及騎車時低頭滑手機問題，因此對比於抬頭顯示器，我們就想到用「低頭顯示器」這樣的概念來突顯本專題的應用目的。

本專題「低頭顯示器–道路交通號誌提醒系統」深度整合資訊科的核心課程，透過實際問題的解決，將理論轉化為應用。具體關聯如下：

一、從理論到硬體

我們將「基本電學」與「數位邏輯」的基礎，應用在 ESP32 控制器與 LED 電路設計中，讓原本靜態的電路圖變成動態的交通號誌。

二、從邏輯到軟體

透過「軟體設計實習」、「可程式邏輯」課程所鍛鍊的程式邏輯，我們成功撰寫出精準的倒數廣播程式，體會到軟硬體協作的深度與挑戰。

三、從技術到智慧城市

利用「通訊技術」中的 BLE 廣播特性，我們實現了免配對的即時資訊傳輸，這正是物聯網（IoT）在現代智慧交通中的核心應用。

四、從實作到社會關懷

研究將專業技術與交通安全議題結合，學生在完成專題的過程中，除了提升實務能力，更能理解科技如何服務於人，實現跨領域學習的最高價值。

參、創意發想與設計過程

一、創意發想

有一次坐在家人開的車子裡，駕駛人在停等紅燈時習慣滑一下手機，因為太專心了以致於紅燈已轉變成綠燈還沉浸在手機裡，結果後面車子看到我們車子綠燈了還不往前開，就大按喇叭「叭~~」，好大一聲嚇壞眾人，但由於我們理虧，就很不好意思的趕快往前開離現場。

由這次經驗，以及其他組員也看過路上更多機車騎士在停等紅燈時幾乎都在滑手機，也經常滑到忘我而忽略了交通號誌的燈號轉換，令人覺得

非常不安全，所以團隊就想到如果能開發一套系統，可以由紅綠燈端傳送訊號到駕駛人的手機，手機 App 會顯示紅燈倒數秒數，而在設定倒數 10 秒時 App 會發出提示音效，提醒駕駛人紅燈即將結束，準備轉換成綠燈後啟動開車，這就是我們創意發想的歷程。

二、設計過程

我們在確立創意專題題目後，就著手進行相關設計，為完成本專題需達成以下關鍵功能

(一)手機端藍芽接收不需配對

我們在網路上查詢手機端可以不需配對的通訊技術，主要有 NFC、BLE 廣播及 Wi Fi P2P 等技術，而 NFC 的傳輸距離只能是十公分左右的短距離通訊，BLE 廣播的傳輸距離可達幾公尺遠，BLE 是低功耗藍芽傳輸的簡稱，藍芽裝置以「廣播模式」傳輸資訊，手機只需掃描訊號即可接收，不需配對，因此選擇以 BLE 功能達成通訊目的。

(二)控制成本達成系統需求

在確定以 BLE 廣播通訊為本專題之核心技術後，需決定以哪一種控制器來實現專題功能，因我們在單晶片實習課程中主要學習 Arduino 微控制器，對於 Arduino 程式開發最為熟悉，進而查詢到 ESP32 微控制器本身內含 BLE 藍芽功能，且相關開發套件容易取得，因此決定以 ESP32 微控制器為系統開發核心。

(三)手機 App 程式設計

團隊成員曾參加過全國技能競賽行動應用開發職類比賽，對於以 Android 行動裝置 App 程式設計有很高度的技能，因此手機端 App 程式開發的技術不是以目前技高課程中所教的 app inventor 來做，而是以 Android Studio 完成，更能達成專題能在背景運作的功能。

肆、設計相關原理

如圖 2 所示為本專題之系統架構圖，以 ESP32 為控制核心，控制模擬路口雙向紅綠燈及顯示秒數，採用 BLE 無線傳輸，不需與行動裝置進行配對，即時將紅燈剩餘秒數傳送至行動裝置的 App，達到提醒駕駛人注意紅綠燈切換之目的。

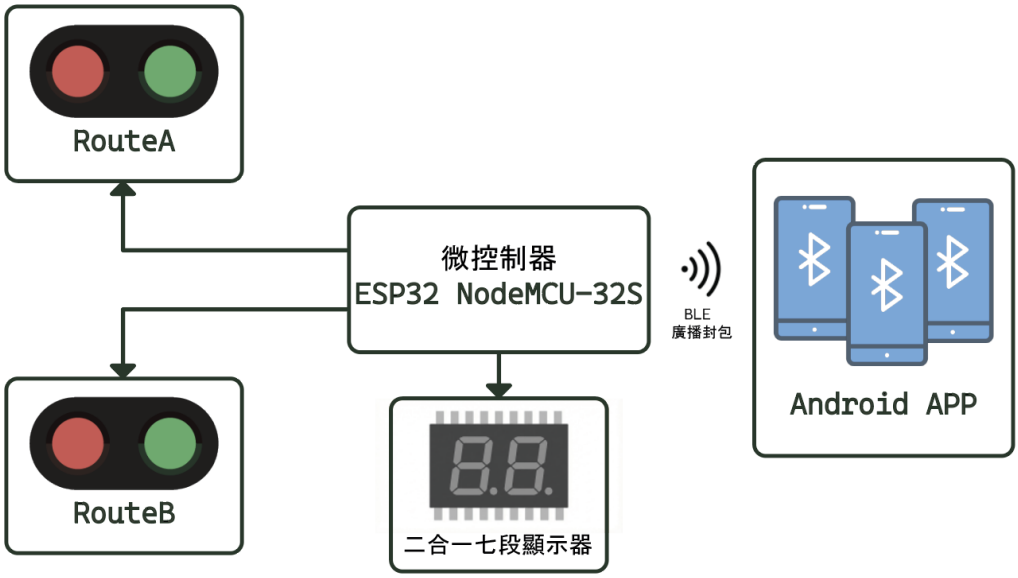


圖 2 系統架構圖

研究透過建構硬體電路、撰寫系統控制程式，並結合行動裝置 BLE 無線通訊，完成具即時顯示與提醒功能的紅綠燈時間提醒系統。整體研究過程依系統架構分為兩個階段：電子電路設計（硬體）與系統程式設計（通訊與控制）。以下詳細說明各階段的設計原理與實施細節，從初始討論到最終整合的完整流程，強調每個步驟的技術選擇、挑戰與解決方案，讓老師了解我們如何一步步實現系統。

本專題設計原理如下

一、元件清單

(一)微控制器

ESP32 開發板（含 BLE 功能）×1

(二)電源

USB 電源線（用於供電 ESP32 開發板）

(三)輸出元件

1、紅色 LED ×2
2、綠色 LED ×2
3、二合一兩位數七段顯示器 ×1
4、IC7447 ×1

5、自製 3D 列印紅綠燈模型

(四)輔助元件

1、麵包板（Breadboard）×1（用於開發階段）
2、杜邦線（Jumper wires）數條（用於開發與測試連接）
3、16pins IC 腳座 ×1
4、單排排母 數個（用於最終電路板固定 ESP32）
5、電路板
6、OK 線(電路焊接)
7、瓦楞板 30cm×60cm (專題模型基底)

二、電子電路設計（硬體）

階段目標為確保紅綠燈時序精確、顯示穩定、通訊即時。以 ESP32 為核心，模擬十字路口（Route A、Route B）交通情境。供電採用 USB 直接供電 ESP32（穩定 5V 輸入，避免電池波動）。研究過程如下，增加細節如工具使用、測試指標與調整理由，讓老師了解我們如何從概念到實體模型：

(一) 討論與初步設計

討論系統需求，包括紅綠燈互鎖邏輯（避免雙綠燈事故）、BLE 通訊範圍、低頭族安全應用（提前提醒減少分心）。確認使用 ESP32 作為核心控制單元，因為其內建 BLE 模組、低功耗（適合長時運行）與 GPIO（至少 18 個可用），能處理多工任務。決定二合一七段顯示器重複倒數，以精簡硬體成本（減少一組顯示器），用戶依紅綠燈顏色分辨路線。

(二) 插麵包板測試

開發階段使用麵包板與杜邦線組裝原型。連接紅綠 LED、二合一七段顯示器與 IC7447，測試燈號切換（綠燈 30 秒、紅燈 30 秒週期）、倒數顯示。測量指標：同步延遲 < 1s，燈號準確率 100% 調整電阻值確保 LED 亮度均勻，驗證同步機制。

(三)調整與優化

調整 GPIO 配置。確認十字路口安全互鎖：程式邏輯 if (RouteA_green) then RouteB_red，反之亦然。測試 BLE 廣播時顯示穩定，以及解決二合一掃描顯示錯亂問題。

(四)做成模型焊接

轉移至電路板，使用 OK 線（跳線）焊接固定組裝。採用 IC 腳座與單排排母，便於更換元件。自製 3D 列印紅綠燈模型（使用 Tinkercad 設計，PLA 材質列印，高 10 cm，之後內嵌 LED），整合買的道路模型(包含馬路與標線)。最終固定於瓦楞板上（尺寸 30x30 cm，便攜展示），形成完整十字路口模擬。測試整體穩定性。

三、系統程式設計（通訊與控制）

本階段採用多工同步架構，確保 BLE 通訊與號誌控制精確同步。研究過程融入寫程式與調整步驟，增加細節如代碼邏輯、App 後台功能，讓老師了解軟體如何與硬體整合。

(一)討論與規劃

討論 BLE 協定（廣播模式免配對，適合多用戶接收）與 App 功能（後台提醒避免干擾）。決定封包格式與多任務流程，選擇 Jetpack Compose 作為 Android 開發平台，因為它是現代 UI 框架（基於 Kotlin，支援 declarative UI），易於建構動態介面（如倒數動畫），並優化性能（低記憶體使用）。

(二)調整與測試

程式測試中調整廣播間隔（每 1,000 ms，setAdvertisingData()），確保無延遲（測量平均 150 ms）。整合硬體後，反覆除錯同步問題（如使用 Serial.print() 監控變數）。App 測試：模擬 BLE 資料，驗證解析與提醒（if (remaining < threshold) { showNotification();}）。

(三)系統整合

將程式燒錄至 ESP32，與硬體模型結合。確認 App 接收資料後觸發提醒，測試整體：連續 10 週期，準確率 100%。

四、BLE 通訊協定與封包設計

(一)通訊模組

使用 ESP32 內建 BLE 類別函式

```
BLE Server *pServer=BLEDevice::createServer();
```

(二)廣播方式

自訂封包 / Characteristic 發送即時路口資訊（pCharacteristic->setValue(data); pAdvertising->start();）。

(三)資料封包格式

Byte 0 / Byte 1：製造商測試 ID（固定值，如 0xFF, 0x01，用於 App 識別）。
Byte 2：路線 A 的紅綠燈狀態 (0x00: 紅燈、0x01: 綠燈)。
Byte 3：路線 B 的紅綠燈狀態 (0x00: 紅燈、0x01: 綠燈)。
Byte 4：剩餘秒數（30 秒綠燈倒數）。

(四)更新頻率

每 1,000 毫秒同步（使用 millis()計算經過時間）。

(五)設計目的

封包簡潔（僅 5 Bytes），節省功耗（廣播模式低電），降低延遲（<200 ms），確保行動端資料一致（App 解析 byteArray.get(2)等）。

(六)多任務同步控制流程

1、軟體架構：
中斷服務程式（ISR） + 多工同步（void onTimer() { updateSeconds(); }）。
2、核心控制任務：
每秒觸發中斷，計算秒數（remaining--； if (remaining == 0) switchLight();），切換 LED。
3、顯示與通訊任務：
中斷後更新 BCD 顯示（digitalWrite(pins, bcdValue);）與 BLE Characteristic。
4、設計目的：
零誤差同步（ISR 優先級高，避免 delay 阻塞）。

(七)Android 行動應用程式開發與解析

開發平台：Jetpack Compose（Google 官方 UI 框架，優勢：declarative 程式碼簡潔，如 Text(text = "剩餘: $seconds")；支援動態更新與後台；比傳統 XML 佈局更高效。
1、裝置掃描與選取：
自動篩選特定 UUID（BleManager.startScan()），使用者 ExposedDropdownMenuBox 選 Route A/B（onItemSelected 監聽）。
2、數據訂閱：
透過 Notification 主動接收資料（getManufacturerSpecificData() ），降低手機功耗（無需輪詢）。

```
接受資料邏輯
private val scanCallback = object : ScanCallback() {
        override fun onScanResult(callbackType: Int, result: ScanResult?) {
            result?.scanRecord ?: return

            if (!isDeviceFound) {
                isDeviceFound = true
                onStatusChanged?.invoke("找到裝置，接收資料中...")
                Log.d("BleManager", "Device found: ${result.device.address}")
            }

            // The data is in the manufacturer specific data field.
            // The key is the manufacturer ID (0xFFFF).
            val manufacturerData =
result.scanRecord!!.getManufacturerSpecificData(MANUFACTURER_ID)

            if (manufacturerData != null) {
                // The returned data is the payload *after* the ID.
                // ESP32 payload: [0xFF, 0xFF, routeA, routeB, countdown]
                // We get: [routeA, routeB, countdown] (3 bytes)
                onDataReceived?.invoke(manufacturerData)
            }
        }
```

圖 3　Android 接收程式解析架

3、資料解析：

將 Raw Bytes 轉換為整數（val statusA = data[2].toInt()；val seconds = data[4].toInt()），解讀燈號顏色與剩餘秒數，用於語音/視覺提醒（MediaPlayer.play()；Composable 動如 AnimatedContent）。

五、使用者操作與應用流程

為提升系統實用性與操作便利性，本研究規劃完整的使用者操作流程，說明如下：

(一) 啟動

使用者起開啟 Android App，按下監測即可半自動掃描附近的 BLE 訊號。

(二) 設定

使用者選擇監控之行進方向（Route A 或 Route B）。設定提醒閾值（例如剩餘 10 秒時提醒）。

(三) 監測及提醒

當提醒時間及路線到達使用者設定值時，螢幕彈出通知及播放提醒音效。

(四)模型

同一時間，硬體端模型的七段顯示器與紅綠燈模組同步顯示倒數時間與燈號狀態，確保系統整體一致性。

伍、作品功用與操作方式

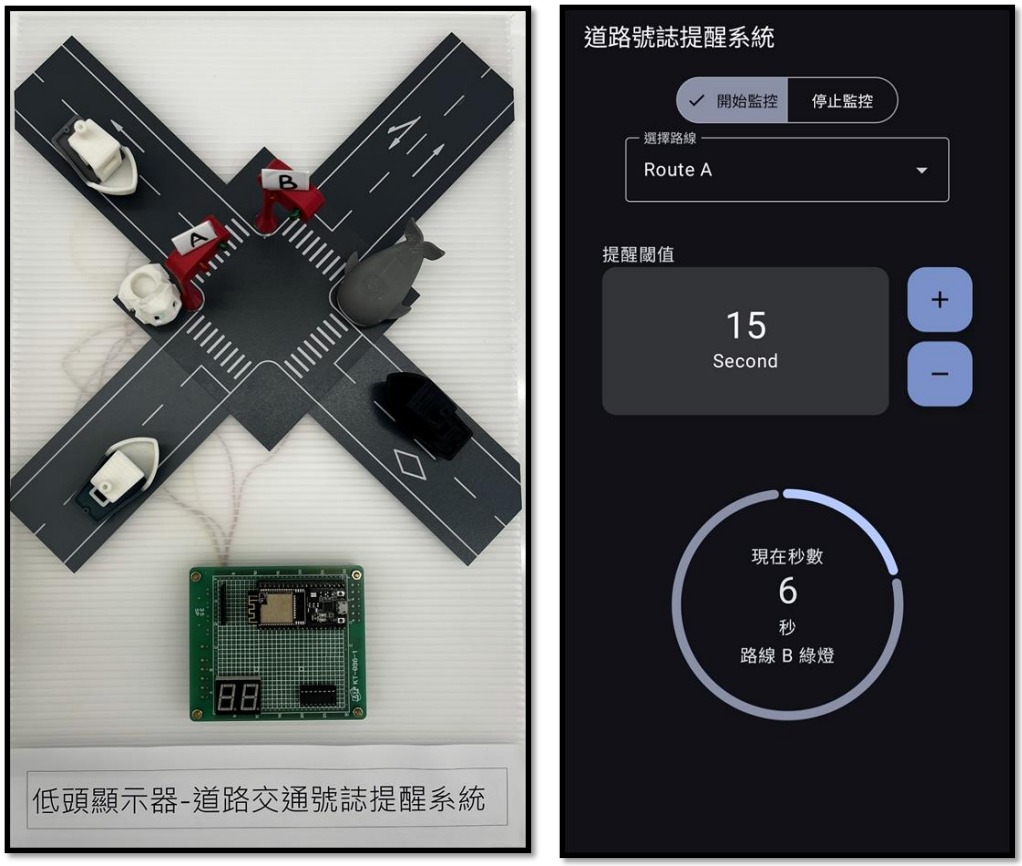


圖 4 作品硬體及軟體展示

本專題完成之硬體模擬架構如圖 4 左圖，手機 App 軟體設計使用者介面如圖 4 右圖，現就本專題之功用及操作方式說明如下

一、硬體模擬架構

我們以瓦楞板架設模擬十字路口場景，以 3D 列印交通號誌燈桿，為區分雙向路口，以編號 Route A 及 Route B 標示為不同方向路口號誌，電路板部分有一兩位數七段顯示器，由控制器控制顯示紅綠燈剩餘秒數，若以 Route A 先進入紅燈時，則 Route B 為綠燈，此時的七段顯示器顯示秒數為雙向紅綠燈剩餘秒數，以共同顯示為簡化模擬設計。

二、手機 App 功能

手機 App 使用者介面可設定提醒閥值，即讓使用者設定紅燈倒數轉換成綠燈前幾秒時,App 會發出提示聲響,手機 App 接收由紅綠燈端的 ESP32 控制器所發出的 BLE 廣播封包,BLE 通訊的特色是手機藍芽接收端不需事先配對，當手機接收到廣播封包時，App 可解析 ESP32 所傳送過來的紅燈倒數秒數，將倒數秒數同步顯示在 App 使用者介面。

陸、製作歷程說明

經由團隊成員從創意發想，到聚焦題目構思，確立研究主題後，著手蒐集資料與探討研究方法，並與指導老師討論技術設計可行性評估後，進行專題製作，以下為本專題之製作歷程：

一、硬體功能測試

先以 ESP32 開發板進行硬體功能測試，依據系統架構圖，我們的電路需要兩組紅綠燈交互顯示，以及一組兩位數七段顯示器共用顯示雙向紅綠燈剩餘秒數

(一)紅綠燈模擬與同步測試

模擬週期設定：Route A 綠燈 30 秒、紅燈 30 秒；Route B 與 Route A 互鎖，當 Route A 為綠燈時 Route B 為紅燈。

(二)顯示驅動測試

二合一七段顯示器與 IC7447 組合:輸入 BCD 碼後,顯示剩餘秒數,我們以 30 秒表示紅綠燈倒數剩餘秒數。

(三)組裝階段驗證

開發階段：使用麵包板與杜邦線快速原型，連接穩定，無短路，如圖 5 所示。

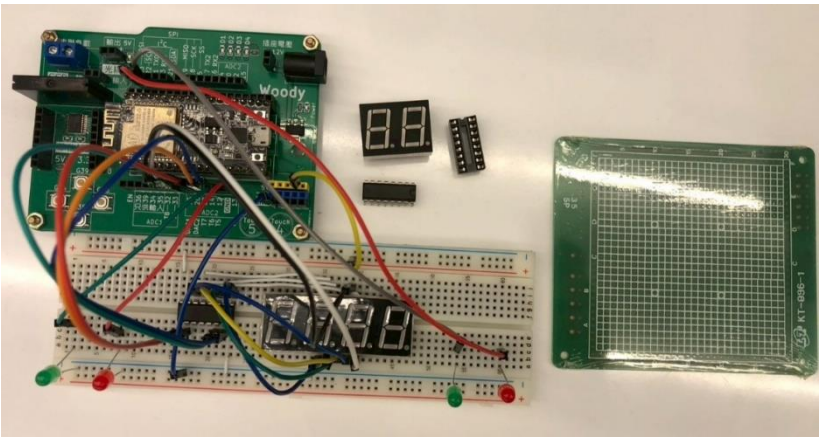


圖 5 電路接線測試

二、電路設計及焊接

以 kicad 電路軟體設計電路，並將零件焊接於萬用電路板，如圖 6 所示。

圖 6 完成電路設計及焊接

三、手機 App 程式設計與軟體測試

以 Android Studio 開發本專題之 App 程式，使用 Jetpack Compose 設計 UI 元件，以 Kotlin 程式語言完成 BLE 廣播封包解析，並能讓程式在背景環境執行，使能應用於車輛停等紅燈時接收 BLE 廣播封包之需求，我們也將此程式開發為 IOS 版本，因此能將本專題擴及 iphone 與 Andriod 兩大群體使用者，如圖 7 及 8 所示。

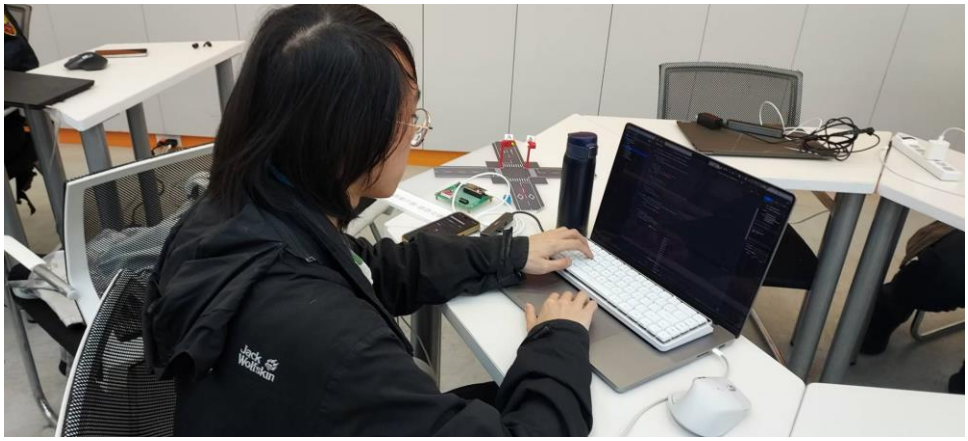


圖 7 手機 App 程式開發

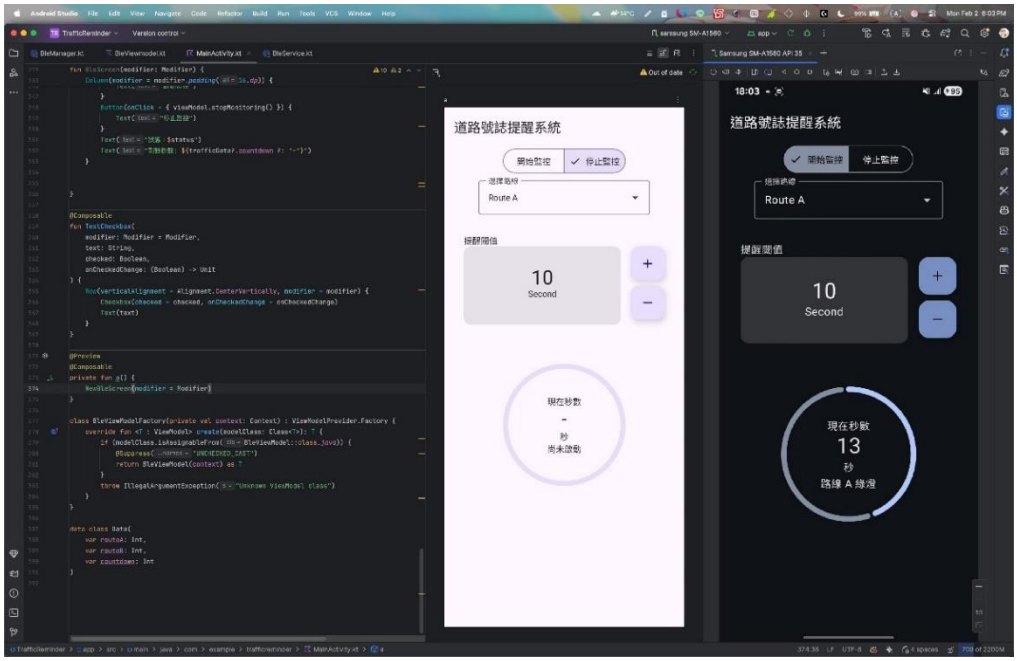


圖 8 Android 程式開發畫

四、作品平台整合

為將軟硬體整合於作品平台展示，我們採購十字路口道路模型及瓦楞板，以 Tinkercad 3D 設計軟體設計模擬交通號誌燈桿，並以 3D 印表機列印燈桿兩座，在燈桿上方可嵌入紅色及綠色 LED 兩只，燈桿採中空設計，內部需能容納導線連接 LED 至微控制器，如圖 9 及 10 所示。

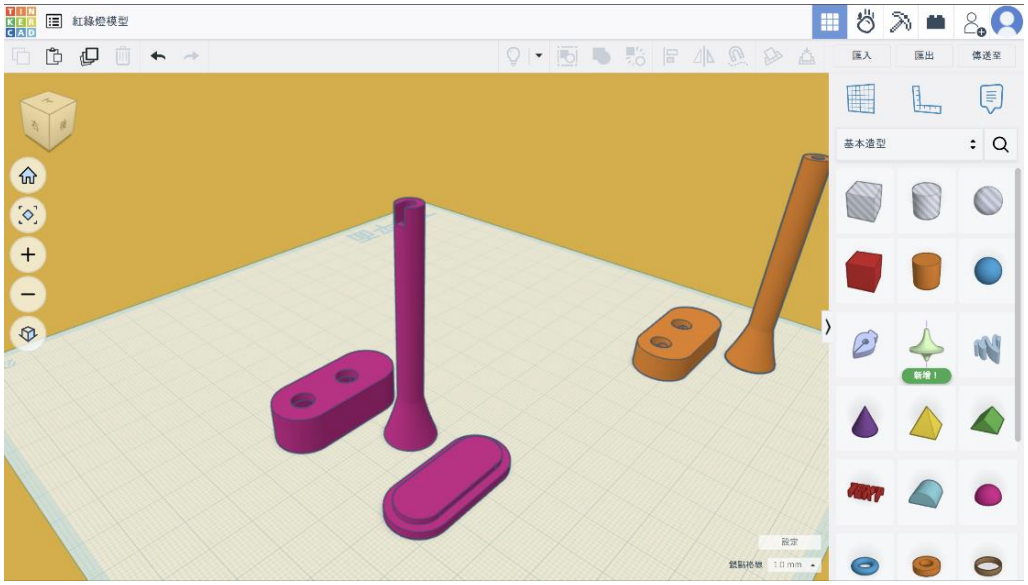


圖 9 號誌燈桿 3D 設計

圖 10 號誌燈桿 3D 列印

作品整合平台完成如圖 11 所示。

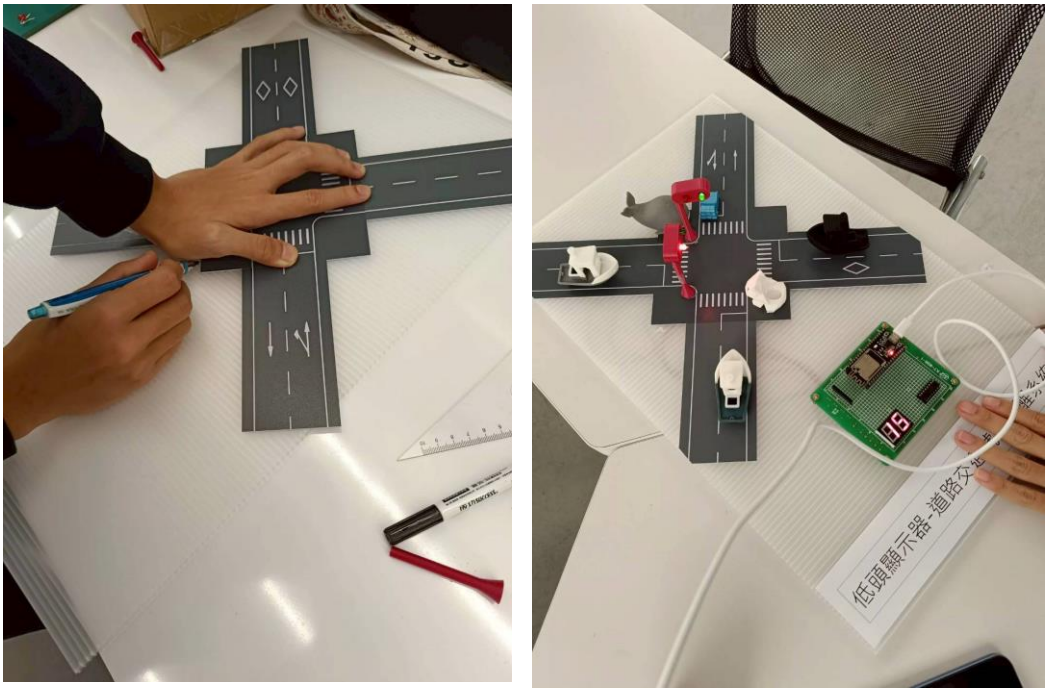


圖 11 硬體架構平台組裝完成

研究成功建構並測試智慧道路號誌提醒系統，透過硬體模擬與軟體整合，驗證系統的實時性、穩定性與提醒效能。

柒、附錄

一、作品分工表

參賽學生	工作任務
A	電路板程式與廣播、功能測試、模型設計製作、報告製作、影片
B	ESP32 測試實裝、電路焊接、功能測試、報告製作
C	BLE 通訊、App 架構、報告製作

二、競賽日誌

年	月	日	進度	紀錄	工作分配
114	10	24	討論專題主題與功能規劃	地點：教室 器材:黑板、筆 時數：2 小時	同學 A：準備器材 同學 B：查資料 同學 C：借場地
114	10	31	購買材料	地點：電子材料行 器材： 時數：3 小時	同學 A:列出所需硬體清單 同學 B:查詢規格與價格 同學 C:負責採購與收貨確認
114	11	7	繪製電路圖、麵包版試做	地點：學校 器材：紙筆、杜邦線、麵包版、ESP32、LED 時數：8 小時	同學 A：繪製電路圖 同學 B:麵包板接線與測試 同學 C:記錄電路配置與測試結果
114	11	14	撰寫 ESP32 韌體程式	地點：學校 器材：電腦、ESP32 時數：4 小時	同學 A:撰寫主要控制邏輯 同學 B：測試與除錯 同學 C:BLE 通訊程式撰寫
114	11	21	撰寫 Android App 程式	地點：學校 器材：電腦、Android 手機 時數：4 小時	同學 A:BLE 資料解析 同學 B：App 測試 同學 C:App 架構設計
114	11	28	ESP32 與硬體整合	地點：學校 器材:ESP32、電腦 時數：4 小時	同學 A：硬體組裝 同學 B：系統測試 同學 C：系統測試

114	12	5	軟硬體結合(BLE 通訊)	地點：學校 器材：電腦、手機、ESP32 時數：4 小時	同學 A：BLE 傳送端（ESP32）調整 同學 B：通訊穩定度測試 同學 C：BLE 接收端（Android）處理
114	12	12	製作模型	地點：學校 器材：雙面膠、道路模型、3D 列印機 時數：4 小時	同學 A：模型設計 同學 B：組裝與固定 同學 C：外觀整理
114	12	19	整合測試	地點：學校 器材：電腦、手機、模型、ESP32 時數：5 小時	同學 A：功能測試 同學 B：壓力與穩定性測試 同學 C：測試紀錄整理
114	12	26	製作報告	地點：學校 器材：電腦 時數：3 小時	同學 A：撰寫系統說明 同學 B：整理流程與圖表 同學 C：校稿與排版